

务驱动的变体中，费舍尔信息的计算十分耗时。但总体而言，Child-tuning 可以改善大语言模型在多种下游任务中的表现，尤其是在训练数据有限的情况下。此外，Child-tuning 可以很好地与其他 PEFT 方法的集成，进一步提升模型性能。

除 Child-tuning 外，还有一些其他基于学习的参数选择方法。例如，Zhao 等人 [55] 引入与模型权重相关的二值矩阵来学习，通过阈值函数生成参数掩码 (mask)，然后在反向传播过程中通过噪声估计器进行更新。类似 FishMask，Fish-Dip [5] 也使用 Fisher 信息来计算掩码，但掩码将在每个训练周期动态重新计算。LT-SFT [2] 受“彩票假设” [11] 启发，根据参数重要性，根据在初始微调阶段变化最大的参数子集形成掩码。SAM [12] 提出了一个二阶逼近方法，通过解析求解优化函数来帮助决定参数掩码。

基于选择的方法通过选择性地更新预训练模型的参数，在保持大部分参数不变的情况下对模型进行微调。基于选择的方法能够显著减少微调过程中所需要更新的参数，降低计算成本和内存需求。对于资源受限的环境或者需要快速适应新任务的场景尤其适用。然而，这些方法也面临挑战，比如，如何选择最佳参数子集，以及如何平衡参数更新的数量和模型性能之间的关系。

4.4 低秩适配方法

低维固有维度假设 [1] 表明：过参数化模型的固有维度是很低的；换言之，存在可以与全参数更新媲美的低维的参数更新。基于这一假设，低秩适配方法 (Low-rank Adaptation Methods) 通过低秩矩阵来近似原始权重更新矩阵，并仅微调低秩矩阵，以大幅降低模型参数量。在本节中，我们首先将介绍最经典的低秩适配方法 LoRA 的实现细节并分析其参数效率。接着，将介绍 LoRA 的相关变体。最后，介绍基于 LoRA 插件化特性，以及其任务泛化能力。

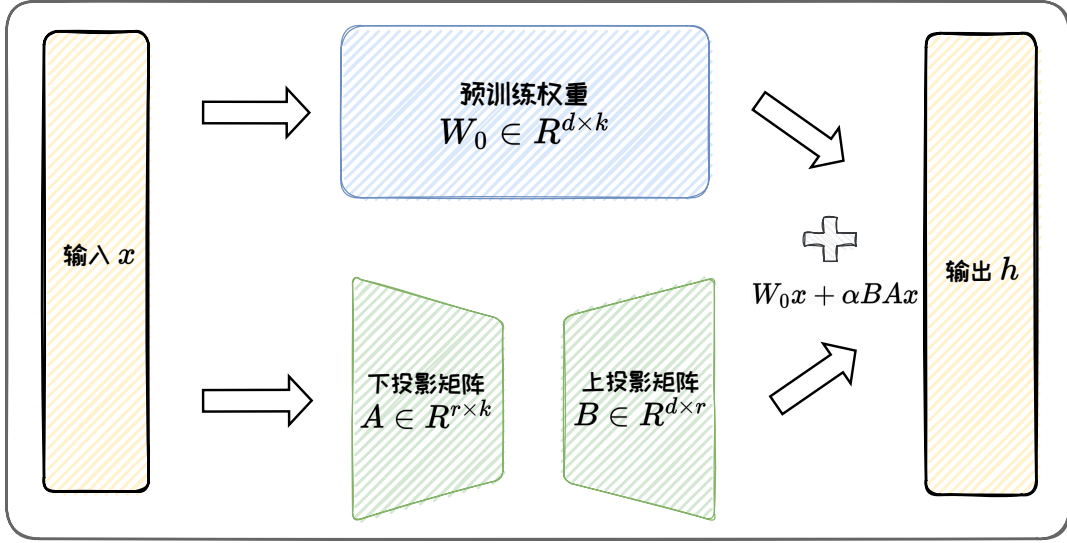


图 4.8: LoRA 示意图。

4.4.1 LoRA

低秩适配 (Low-rank Adaptation, LoRA) [19] 提出利用低秩矩阵近似参数更新矩阵来实现低秩适配。该方法将参数更新矩阵低秩分解为两个小矩阵。在微调时, 通过微调这两个小矩阵来对大语言模型进行更新, 大幅节省了微调时的内存开销。本小节首先介绍 LoRA 方法的具体实现过程, 然后分析其计算效率。

1. 方法实现

给定一个密集神经网络层, 其参数矩阵为 $W_0 \in \mathbb{R}^{d \times k}$, 为适配下游任务, 我们通常需要学习参数更新矩阵 $\Delta W \in \mathbb{R}^{d \times k}$, 对原始参数矩阵进行更新 $W = W_0 + \Delta W$ 。对于全量微调过程, ΔW 是需对该层所有 $d \times k$ 个参数计算梯度, 这通常需要大量的 GPU 内存, 成本高昂。为解决这一问题, 如图 4.8, LoRA 将 ΔW 分解为两个低参数量的矩阵 $B \in \mathbb{R}^{d \times r}$ 和 $A \in \mathbb{R}^{r \times k}$, 使得更新过程变为:

$$W = W_0 + \alpha B A, \quad (4.9)$$

其中, 秩 $r \ll \min\{d, k\}$, B 和 A 分别用随机高斯分布和零进行初始化, α 是缩放

因子，用于控制 LoRA 权重的大小。在训练过程中，固定预训练模型的参数，仅微调 B 和 A 的参数。因此，在训练时，LoRA 涉及的更新参数数量为 $r \times (d + k)$ ，远小于全量微调 $d \times k$ 。实际上，对于基于 Transformer 的大语言模型，密集层通常有两种类型：注意力模块中的投影层和前馈神经网络（FFN）模块中的投影层。在原始研究中，LoRA 被应用于注意力层的权重矩阵。后续工作表明将其应用于 FFN 层可以进一步提高模型性能 [14]。

LoRA 仅微调部分低秩参数，因此具有很高的参数效率，同时不会增加推理延迟 [10]。此外，低秩矩阵还可以扩展为低秩张量 [3]，或与 Kronecker 分解结合使用，以进一步提高参数效率 [9, 16]。除了参数效率外，在训练后可以将 LoRA 参数与模型参数分离，所以 LoRA 还具有可插拔性。LoRA 的可插拔特性使其能够封装为被多个用户共享和重复使用 [38] 的插件。当 we 有多个任务的 LoRA 插件时，可以将这些插件组合在一起，以获得良好的跨任务泛化性能 [20]。我们将在 4.4.3 提供具体案例详细介绍基于 LoRA 插件的任务泛化。

2. 参数效率

下面我们以一个具体的案例分析 LoRA 的参数效率。在 LLaMA2-7B [40] 模型中微调第一个 FFN 层的权重矩阵为例，全量微调需要调整 $11,008 \times 4,096 = 45,088,768$ 个参数。而当 $r = 4$ 时，LoRA 只需调整 $(11,008 \times 4) + (4 \times 4,096) = 60,416$ 个参数。对于这一层，与全量微调相比，LoRA 微调的参数不到原始参数数量的千分之一。具体来说，模型微调的内存使用主要涉及四个部分：

- 权重内存（Weight Memory）：用于存储模型权重所需的内存；
- 激活内存（Activation Memory）：前向传播内存时中间激活带来的显存占用，主要取决于 batch size 大小以及序列长度等；
- 梯度内存（Gradient Memory）：在反向传播期间需要用来保存梯度的内存，这些梯度仅针对可训练参数进行计算；

- 优化器内存 (Optimization Memory)：用于保存优化器状态的内部存在。例如，Adam 优化器会保存可训练参数的“一阶动量”和“二阶动量”。

文献 [34] 提供在 LLaMA2-7B 模型上使用批量大小为 1, 单个 NVIDIA RTX4090 (24GB) GPU 上进行全量微调和 LoRA 微调的实验对比。根据实验结果，全量微调大约需要 60GB 显存，超出 RTX4090 的显存容量。相比之下，LoRA 只需要大约 23GB 显存。LoRA 显著减少了显存使用，使得在单个 NVIDIA RTX4090 上进行 LLaMA2-7B 微调成为可能。具体来说，**由于可训练参数较少，优化器内存和梯度内存分别减少了约 25GB 和 14GB**。另外，虽然 LoRA 引入了额外的“增量参数”，导致**激活内存和权重内存略微增加（总计约 2GB）**，但考虑到整体内存的减少，这种增加是可以忽略不计的。此外，减少涉及到的参数计算可以加速反向传播。与全量微调相比，LoRA 的速度提高了 1.9 倍。

4.4.2 LoRA 相关变体

虽然 LoRA 在一些下游任务上能够实现较好的性能，但在许多复杂的下游任务（如数学推理 [4, 6, 54]）上，LoRA 与全量微调之间仍存在性能差距。为弥补这一差距，许多 LoRA 变体方法被提出，以进一步提升 LoRA 在下游任务中的适配性能。现有方法主要从以下几个角度进行改进 [31]：(1) 打破低秩瓶颈；(2) 动态秩分配；(3) 训练过程优化。接下来，将分别介绍这三种类型变种的代表性方法。

1. 打破低秩瓶颈

LoRA 的低秩更新特性使其在参数效率上具有优势；然而，这也限制了大规模语言模型记忆新知识和适应下游任务的能力 [4, 13, 21, 54]，即存在低秩瓶颈。Biderman 等人 [4] 的实验研究表明，全量微调的秩显著高于 LoRA 的秩（10-100 倍），并且增加 LoRA 的秩可以缩小 LoRA 与全量微调之间的性能差距。因此，一些方法被提出，旨在打破低秩瓶颈 [25, 36, 48]。

例如，ReLoRA [25] 提出了一种合并和重置（merge-and-reinit）的方法，该方法在微调过程中周期性地将 LoRA 模块合并到大语言模型中，并在合并后重新初始化 LoRA 模块和优化器状态。具体地，合并的过程如下：

$$W^i \leftarrow W^i + \alpha B^i A^i, \quad (4.10)$$

其中， W^i 是原始的权重矩阵， B^i 和 A^i 是低秩分解得到的矩阵， α 是缩放因子。合并后，将重置 B^i 和 A^i 的值重置，通常 B^i 会使用特定的初始化方法（如 Kaiming 初始化）重新初始化，而 A^i 则被设置为零。为了防止在重置后模型性能发散，ReLoRA 还会通过幅度剪枝对优化器状态进行部分重置。合并和重置的过程允许模型在保持总参数量不变的情况下，通过多次低秩 LoRA 更新累积成高秩状态，从而使得 ReLoRA 能够训练出性能接近全秩训练的模型。

2. 动态秩分配

然而，LoRA 的秩并不总是越高越好，冗余的 LoRA 秩可能会导致性能和效率的退化。并且，微调时权重的重要性可能会因 Transformer 模型中不同层而存在差异，因此需要为每个层分配不同的秩 [7, 30, 41, 52]。

例如，AdaLoRA [52] 通过将参数更新矩阵参数化为奇异值分解（SVD）的形式，再通过奇异值剪枝动态调整不同层中 LoRA 模块的秩。具体地，AdaLoRA 使用奇异值分解重新表示 ΔW ，即

$$W = W_0 + \Delta W = W_0 + P \Lambda Q, \quad (4.11)$$

其中， $P \in \mathbb{R}^{d \times r}$ 和 $Q \in \mathbb{R}^{r \times k}$ 是正交的， Λ 是一个对角矩阵，其中包含 $\{\lambda_i\}_{1 \leq i \leq r}$ 的奇异值。在训练过程中， W_0 的参数被固定，仅更新 P 、 Λ 和 Q 的参数。根据梯度权重乘积大小的移动平均值构造奇异值的重要性得分，对不重要的奇异值进行迭代剪枝。此外，为了增强稳定训练性，AdaLoRA 引入一个额外的惩罚项确保 P

和 Q 之间的正交性:

$$R(P, Q) = \|P^T P - I\|_F^2 + \|Q Q^T - I\|_F^2, \quad (4.12)$$

其中, I 是单位矩阵, $\|\cdot\|_F$ 代表 Frobenius 范数。

3. 训练过程优化

在实际微调过程中, LoRA 的收敛速度比全量微调要慢。此外, 它还对超参数敏感, 并且容易过拟合。这些问题影响了 LoRA 的效率并阻碍了其下游适配性能。为了解决这些问题, 一些工作尝试对 LoRA 的训练过程进行优化 [32, 45]。代表性方法 DoRA (权重分解低秩适应) [29] 提出约束梯度更新, 侧重于更新参数的方向变化。它将预训练权重 $W_0 \in \mathbb{R}^{d \times k}$ 分解为方向和大小两个组件, 并仅将 LoRA 应用于方向组件以增强训练稳定性。具体地, DoRA 将 $W_0 \in \mathbb{R}^{d \times k}$ 重新表示为:

$$W_0 = m \frac{V}{\|V\|_c} = \|W_0\|_c \frac{W_0}{\|W_0\|_c}, \quad (4.13)$$

其中, $m \in \mathbb{R}^{1 \times k}$ 是大小向量, $V \in \mathbb{R}^{d \times k}$ 是方向矩阵, $\|\cdot\|_c$ 是矩阵在每一列上的向量范数。随后, DoRA 仅对方向矩阵 V 施加 LoRA 进行参数化, 定义为:

$$W' = m \frac{V + \underline{\Delta V}}{\|V + \underline{\Delta V}\|_c} = m \frac{W_0 + \underline{BA}}{\|W_0 + \underline{BA}\|_c}, \quad (4.14)$$

其中, ΔV 是由 LoRA 学习的增量方向更新, 下划线参数表示可训练参数。

4.4.3 基于 LoRA 插件的任务泛化

在 LoRA 微调结束后, 我们可以将参数更新模块 B 和 A 从模型上分离出来, 并封装成参数插件。这些插件具有即插即用、不破坏原始模型参数和结构的优良性质。我们可以在不同任务上训练的各种 LoRA 模块, 将这些模块插件化地方式保存、共享与使用。此外, 我们还可以将多个任务的 LoRA 插件组合, 然后将不同任务的能力迁移到新任务上。LoRAHub [20] 提供了一个可用的多 LoRA 组合的方法框架。其可将已有任务上得到的 LoRA 插件进行组合, 从而获得解决新任务的能力。